▲

Web Interface Guidelines

Interfaces succeed because of hundreds of choices. This is a living, non-exhaustive list of those decisions. Most guidelines are framework-agnostic, some specific to React/Next.js. Feedback is welcome.

Interactions

- **Keyboard works everywhere.** All flows are keyboard-operable & follow the WAI-ARIA Authoring Patterns.
- **Clear focus.** Every focusable element shows a visible, unobscured focus ring. Prefer `:focus-visible` over `:focus` to avoid distracting pointer users. Set `:focus-within` for grouped controls. Sticky headers, footers, banners, & overlays never cover the focused element.
- **Manage focus.** Use focus traps, move & return focus according to the WAI-ARIA Patterns.
- **Match visual & hit targets.** Exception: if the visual target is < 24px, expand its hit target to ≥ 24px. On mobile, the minimum size is 44px.
- **Mobile input size.** `<input>` font size is ≥ 16px on mobile to prevent iOS Safari auto-zoom/pan on focus. Or set `<meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1" />`.
- **Respect zoom.** Never disable browser zoom.
- **Hydration-safe inputs.** Inputs must not lose focus or value after hydration.
- **Don't block paste.** Never disable paste in `<input>` or `<textarea>`.
- **Loading buttons.** Show a loading indicator & keep the original label.
- **Minimum loading-state duration.** If you show a spinner/skeleton, add a short show-delay (~150–300 ms) & a minimum visible time (~300–500 ms) to avoid flicker on fast responses. The `<Suspense>` component in React does this automatically.
- **URL as state.** Persist state in the URL so share, refresh, Back/Forward navigation work e.g., nuqs.
- **Optimistic updates.** Update the UI immediately when success is likely; reconcile on server response. On failure, show an error & roll back or provide Undo.
- **Ellipsis for further input & loading states.** Menu options that open a follow-up e.g., "Rename…" & loading/processing states e.g., "Loading…", "Saving…", "Generating…" end with an ellipsis.
- **Confirm destructive actions.** Require confirmation or provide Undo with a safe window.
- **Prevent double-tap zoom on controls.** Set `touch-action: manipulation`.
- **Tap highlight follows design.** Set `webkit-tap-highlight-color`.
- **Design forgiving interactions.** Controls minimize finickiness with generous hit targets, clear affordances, & predictable interactions, e.g., prediction cones.
- **Tooltip timing.** Delay the first tooltip in a group; subsequent peers have no delay.
- **Overscroll behavior.** Set `overscroll-behavior: contain` intentionally e.g., in modals/drawers.
- **Desktop rubber-banding.** To suppress rubber-band bounce, set `overscroll-behavior: none` on `<html>` inside `@media (pointer: fine)`. Keep page overscroll defaults on coarse-pointer devices to preserve mobile pull-to-refresh. `pointer` describes the primary input, so test hybrid touch devices separately. `none` also disables scroll chaining & native swipe navigation.
- **Scroll positions persist.** Back/Forward restores prior scroll.
- **Autofocus for speed.** On desktop screens with a single primary input, autofocus. Rarely autofocus on mobile because the keyboard opening can cause layout shift.
- **No dead zones.** If part of a control looks interactive, it should be interactive. Don't leave users guessing where to interact.
- **Deep-link everything.** Filters, tabs, pagination, expanded panels, anytime `useState` is used.
- **Clean drag interactions.** Disable text selection & apply `inert` (which prevents interaction) while an element is dragged so selection/hover don't occur simultaneously.
- **Gestures have alternatives.** Every drag, swipe, pinch, or path-based action also works with a tap/click control & keyboard unless the gesture is essential.
- **Links are links.** Use `<a>` or `<Link>` for navigation so standard browser behaviors work (Cmd/Ctrl+Click, middle-click, right-click to open in a new tab). Never substitute with `<button>` or `<div>` for navigational links.
- **Announce async updates.** Use polite aria-live for toasts & inline validation.
- **Locale-aware keyboard shortcuts.** Internationalize keyboard shortcuts for non-QWERTY layouts. Show platform-specific symbols.

Animations

- **Honor `prefers-reduced-motion`.** Provide a reduced-motion variant.
- **Implementation preference.** Prefer CSS, avoid main-thread JS-driven animations when possible.
  - Preference: CSS > Web Animations API > JavaScript libraries e.g., motion.
- **Compositor-friendly.** Prioritize GPU-accelerated properties (`transform`, `opacity`) & avoid properties that trigger reflows/repaints (`width`, `height`, `top`, `left`).
- **Necessity check.** Only animate when it clarifies cause & effect or when it adds deliberate delight e.g., the northern lights.
- **Easing fits the subject.** Choose easing based on what changes (size, distance, trigger).
- **Interruptible.** Animations are cancelable by user input.
- **Input-driven.** Avoid autoplay except for muted, non-essential loops. Automatically playing motion lasting >5 seconds alongside other content has pause, stop, or hide controls.
- **Correct transform origin.** Anchor motion to where it "physically" starts.
- **Never `transition: all`.** Explicitly list only the properties you intend to animate (typically `opacity`, `transform`). `all` can unintentionally animate layout-affecting properties causing jank.
- **Cross-browser SVG transforms.** Apply CSS transforms/animations to `<g>` wrappers & set `transform-box: fill-box; transform-origin: center;`. Safari historically had bugs with transform-origin on SVG & grouping avoids origin miscalculation.

Layout

- **Optical alignment.** Adjust ±1px when perception beats geometry.
- **Deliberate alignment.** Every element aligns with something intentionally whether to a grid, baseline, edge, or optical center. No accidental positioning.
- **Balance contrast in lockups.** When text & icons sit side by side, adjust weight, size, spacing, or color so they don't clash. For example, a thin-stroke icon may need a bolder stroke next to medium-weight text.
- **Responsive coverage.** Verify on mobile, laptop, & ultra-wide. For ultra-wide, zoom out to 50% to simulate.

- **Respect safe areas.** Account for notches & insets with safe-area variables.
- **No excessive scrollbars.** Only render useful scrollbars; fix overflow issues to prevent unwanted scrollbars. On macOS set "Show scroll bars" to "Always" to test what Windows users would see.
- **Let the browser size things.** Prefer flex/grid/intrinsic layout over measuring in JS. Avoid layout thrash by letting CSS handle flow, wrapping, & alignment.

Content

- **Inline help first.** Prefer inline explanations; use tooltips as a last resort.
- **Stable skeletons.** Skeletons mirror final content exactly to avoid layout shift.
- **Accurate page titles.** `<title>` reflects the current context.
- **No dead ends.** Every screen offers a next step or recovery path.
- **All states designed.** Empty, sparse, dense, & error states.
- **Typographic quotes.** Prefer curly quotes (“ ”) over straight quotes (" ").
- **Avoid widows/orphans.** Tidy rag & line breaks.
- **Tabular numbers for comparisons.** Use `font-variant-numeric: tabular-nums` or a monospace like Geist Mono.
- **Redundant status cues.** Don't rely on color alone; include text labels.
- **Icons have labels.** Convey the same meaning with text for non-sighted users.
- **Don't ship the schema.** Visual layouts may omit visible labels, but accessible names/labels still exist for assistive tech.
- **Use the ellipsis character.** `…` over three periods `...`.
- **Anchored headings.** Set `scroll-margin-top` for headers when linking to sections.
- **Resilient to user-generated content.** Layouts handle short, average, & very long content.
- **Locale-aware formats.** Format dates, times, numbers, delimiters, & currencies for the user's locale.
- **Prefer language settings over location.** Detect language via `Accept-Language` header & `navigator.languages`. Never rely on IP/GPS for language.
- **Shield verbatim content from translation.** Wrap brand names, product names, code tokens, & technical identifiers with `translate="no"` so browser auto-translate leaves them intact.
- **Accessible content.** Set accurate names (`aria-label`), hide decoration (`aria-hidden`) & verify in the accessibility tree.
- **Icon-only buttons are named.** Provide a descriptive `aria-label`.
- **Semantics before ARIA.** Prefer native elements (`button`, `a`, `label`, `table`), before `aria-*`.
- **Headings & skip link.** Hierarchical `<h1–h6>` & a “Skip to content” link.
- **Accessible media.** Caption speech & meaningful sounds, provide transcripts for audio-only content, & describe essential visual information. Decorative media is hidden from assistive tech; media controls are keyboard-operable.
- **Brand resources from the logo.** Right-click the nav logo to surface brand assets for quick access.
- **Non-breaking spaces for glued terms.** Use a non-breaking space ` ` to keep units, shortcuts & names together: `10 MB` → `10 MB`, `⌘ + K` → `⌘ + K`, `Vercel SDK` → `Vercel SDK`. Use `` for no space.

Forms

- **Enter submits.** When a text input is focused, Enter submits if it's the only control. If there are many controls, apply to the last control.
- **Textarea behavior.** In `<textarea>`, ⌘/^+Enter submits; Enter inserts a new line.
- **Labels everywhere.** Every control has a `<label>` or is associated with a label for assistive tech.
- **Label activation.** Clicking a `<label>` focuses the associated control.
- **Submission rule.** Keep submit enabled until submission starts; then disable during the in-flight request, show a spinner, & include an idempotency key.
- **Don't block typing.** Even if a field only accepts numbers, allow any input & show validation feedback. Blocking keystrokes entirely is confusing because the user gets no explanation.
- **Don't pre-disable submit.** Allow submitting incomplete forms to surface validation feedback.
- **No dead zones on controls.** Checkboxes & radios avoid dead zones; the label & control share a single generous hit target.
- **Error placement.** Show errors next to their fields; on submit, focus the first error.
- **Autocomplete & names.** Set `autocomplete` & meaningful `name` values to enable autofill.
- **Spellcheck selectively.** Disable for emails, codes, usernames, etc.
- **Correct types & input modes.** Use the right `type` & `inputmode` for better keyboards & validation.
- **Placeholders signal emptiness.** End with an ellipsis.
- **Placeholder value.** Set placeholder to an example value or pattern e.g., `+1 (123) 456-7890` & `sk-012345679…`
- **Unsaved changes.** Warn before navigation when data could be lost.
- **Password managers & 2FA.** Ensure compatibility & allow pasting one-time codes.
- **Don't trigger password managers for non-auth fields.** For inputs like “Search” avoid reserved names (e.g., password), use `autocomplete="off"` or a specific token like `autocomplete="one-time-code"` for OTP fields.
- **Text replacements & expansions.** Some input methods add trailing whitespace. The input should trim the value to avoid showing a confusing error message.
- **Windows `<select>` background.** Explicitly set `background-color` & `color` on native `<select>` to avoid dark-mode contrast bugs on Windows.

Performance

- **Device/browser matrix.** Test iOS Low Power Mode & macOS Safari.
- **Measure reliably.** Disable extensions that add overhead or change runtime behavior.
- **Track re-renders.** Minimize & make re-renders fast. Use React DevTools or React Scan.
- **Throttle when profiling.** Test with CPU & network throttling.
- **Minimize layout work.** Batch reads/writes; avoid unnecessary reflows/repaints.
- **Network latency budgets.** `POST/PATCH/DELETE` complete in <500ms.
- **Keystroke cost.** Prefer uncontrolled inputs; make controlled loops cheap.
- **Large lists.** Virtualize large lists e.g., virtua or content-visibility: auto.
- **Preload wisely.** Preload only above-the-fold images; lazy-load the rest.
- **No image-caused CLS.** Set explicit image dimensions & reserve space.
- **Preconnect to origins.** Use `<link rel="preconnect">` for asset/CDN domains (with crossorigin when needed) to reduce DNS/TLS latency.
- **Preload fonts.** For critical text to avoid flash & layout shift.
- **Subset fonts.** Ship only the code points/scripts you use via unicode-range (limit variable axes

to what you need) to shrink size.

- **Don't use the main thread for expensive work.** Move especially long tasks to Web Workers to avoid blocking interaction with the page.
- **Video over animated GIF.** Prefer `<video autoplay muted loop playsinline>` for looping animation. Modern video is typically much smaller than GIF. Provide a still alternative & honor `prefers-reduced-motion`.
- **Safari video-as-image.** For short, non-essential loops, add an H.264 MP4 source to `<picture>`. Safari renders the video through its image pipeline; other browsers use the still fallback. Gate the MP4 source with `prefers-reduced-motion`. Use `<video>` when users need playback controls or the animation conveys essential information.

Design

- **Layered shadows.** Mimic ambient + direct light with at least two layers.
- **Crisp borders.** Combine borders & shadows; semi-transparent borders improve edge clarity.
- **Nested radii.** Child radius ≤ parent radius & concentric so curves align.
- **Hue consistency.** On non-neutral backgrounds, tint borders/shadows/text toward the same hue.
- **Accessible charts.** Use color-blind-friendly palettes.
- **Minimum contrast.** Prefer APCA over WCAG 2 for more accurate perceptual contrast.
- **Interactions increase contrast.** `:hover`, `:active`, `:focus` have more contrast than rest state.
- **Browser UI matches your background.** Set `<meta name="theme-color" content="#000000">` to align the browser's theme color with the page background.
- **Set the appropriate color-scheme.** Style the `<html>` tag with `color-scheme: dark` in dark themes so that scrollbars and other device UI have proper contrast.
- **Text anti-aliasing & transforms.** Scaling text can change smoothing. Prefer animating a wrapper instead of the text node. If artifacts persist set `translateZ(0)` or `will-change: transform` to promote to its own layer.
- **Avoid gradient banding.** Fading content to dark colors using css masks can cause banding. Background images can be used instead.

Vercel-specific

These preferences reflect Vercel's brand & product choices. They aren't universal guidelines.

Copywriting

- **Active voice.**
  - Instead of *"The CLI will be installed,"* say *"Install the CLI."*
- **Headings & buttons use Title Case** (Chicago). On marketing pages, use sentence case.
- **Be clear & concise.** Use as few words as possible.
- **Prefer `&` over `and`.**
- **Action-oriented language.**
  - Instead of *"You will need the CLI..."* say *"Install the CLI..."*.
- **Keep nouns consistent.** Introduce as few unique terms as possible.
- **Write in second person.** Avoid first person.
- **Use consistent placeholders.**
  - Strings: `YOUR_API_TOKEN_HERE`. Numbers: `0123456789`.
- **Use numerals for counts.**
  - Instead of *"eight deployments"* say *"8 deployments"*.
- **Consistent currency formatting.** In any given context, display currency with either 0 or 2 decimal places, never mix both.
- **Separate numbers & units with a space.**
  - Instead of `10MB` say `10 MB`.
  - Use a non-breaking space e.g., `10 MB`.
- **Default to positive language.** Frame messages in an encouraging, problem-solving way, even for errors.
  - Instead of *"Your deployment failed,"* say *"Something went wrong—try again or contact support."*
- **Error messages guide the exit.** Don't just state what went wrong—tell the user how to fix it.
  - Instead of *"Invalid API key,"* say *"Your API key is incorrect or expired. Generate a new key in your account settings."* The copy & buttons/links should educate & give a clear action.
- **Avoid ambiguity.** Labels are clear & specific.
  - Instead of the button label *"Continue"* say *"Save API Key"*.

Integrate with Agents

Use these guidelines with AI coding agents. Audit all generated interfaces.

Review Command

Install the `web-design-guidelines` skill to review UI code:

```
npx skills add https://github.com/vercel-labs/agent-skills --skill web-design-guide
```

AGENTS.md

Add AGENTS.md to your project so agents apply these guidelines during generation.

- Download AGENTS.md

Join Vercel

We're hiring people who live for these details. Check out the job postings.

Thanks to Adam, Jimmy, Jonnie, Lochie, Paco, Joe, & Austin for feedback.